Filters in Spring Boot :

> Filters allow us to execute common HTTP-level logic before a request reaches the Spring MVC controller and after the request has been processed.

---

1. Why Do We Need Filters?

A typical request in a Spring Boot application may appear to follow this flow:

```
Client / Browser / Postman
        ↓
Spring Boot Application
        ↓
Controller Method
        ↓
Response
```

In a real application, the request does not directly reach the controller. Before executing the controller, the application may need to perform common tasks such as:

- Logging the request URL
- Checking an authentication token
- Blocking suspicious requests
- Measuring request execution time
- Adding common response headers
- Applying character encoding
- Creating a request or trace ID

Writing this logic inside every controller method leads to repeated code.

```java
@GetMapping("/students")
public List<Student> getStudents(HttpServletRequest request)
{
```

```
        System.out.println(
                "Request URL: " + request.getRequestURI()
        );

        // Authentication check
        // Logging
        // Timing logic

        return studentService.getAllStudents();
    }
```

Although this code works, it creates several design problems:

1. The same infrastructure logic is repeated across multiple APIs.
2. Controller methods become difficult to read.
3. Developers may forget to add the logic to newly created APIs.
4. Controllers stop focusing on their primary responsibility: handling application use cases.

A Filter provides a common place for request-level logic that must apply to multiple endpoints.

---

2. Cross-Cutting Concerns

Tasks that apply across many parts of an application are known as **cross-cutting concerns**.

Common examples include:

- Request logging
- Authentication and authorization checks
- Security checks
- Character encoding
- Request timing
- Audit tracking

- Rate limiting
- Correlation or trace IDs

These concerns are not part of the business logic of a particular controller, service, or repository.

They operate across multiple requests, making Filters suitable when the work belongs to the HTTP or Servlet layer.

---

3. Request–Response Flow in Spring Boot

A simplified request flow in a Spring Boot web application is:

```
Client
  ↓
Servlet Container, such as Tomcat
  ↓
Filters
  ↓
DispatcherServlet
  ↓
Interceptors
  ↓
Controller
  ↓
Service
  ↓
Repository
```

The response returns through the same components in reverse order:

```
Repository
  ↓
Service
  ↓
Controller
  ↓
Interceptors
  ↓
DispatcherServlet
  ↓
Filters
```

```
  ↓
Client
```

Where does a Filter execute?

Filters belong to the **Servlet layer**. They execute before the request reaches Spring MVC's `DispatcherServlet`.

```
Client
  ↓
Tomcat
  ↓
Filter 1
  ↓
Filter 2
  ↓
DispatcherServlet
  ↓
Controller
```

Because a Filter executes before `DispatcherServlet`, it can reject a request before Spring MVC performs controller mapping.

> A Filter operates at a lower web level than a Spring MVC Interceptor.

---

4. What Is a Filter?

A Filter acts like a checkpoint through which an HTTP request and response pass.

It can perform logic:

- Before the request continues
- After downstream request processing returns

Conceptually, a Filter works like this:

```
doSomethingBefore();

chain.doFilter(request, response);
```

```
doSomethingAfter();
```

The call to `chain.doFilter(...)` passes control to the next component in the request-processing chain.

A Filter can:

- Inspect request headers, cookies, query parameters, and other metadata
- Add response headers
- Log request and response details
- Measure total request duration
- Validate authentication information
- Wrap the request or response
- Allow the request to continue
- Stop the request immediately

Because Filters operate at the Servlet layer, they are not limited to controller endpoints. Depending on their registration, they may also apply to paths such as:

```
/error
/favicon.ico
/static/logo.png
Servlet endpoints
Spring MVC endpoints
```

---

5. Filters Come from the Servlet API

Filters are not originally a Spring feature. They are defined by the Servlet specification.

In Spring Boot 3 and modern Jakarta-based applications, the Filter interface is available from:

```
jakarta.servlet.Filter
```

A simplified version of the interface looks like this:

```
public interface Filter {

    default void init(FilterConfig filterConfig)
            throws ServletException {
    }

    void doFilter(
            ServletRequest request,
            ServletResponse response,
            FilterChain chain
    ) throws IOException, ServletException;

    default void destroy() {
    }
}
```

The Filter lifecycle contains three methods:

Method	Purpose
`init()`	Called when the Filter is initialized
`doFilter()`	Called when a matching request passes through the Filter
`destroy()`	Called when the Filter is removed or the application shuts down

In most Spring Boot applications, custom Filter logic is mainly written inside `doFilter()`.

---

6. Spring Boot 2 vs Spring Boot 3 Imports

Spring Boot 2 applications generally used the `javax.servlet` package:

```
import javax.servlet.Filter;
import javax.servlet.FilterChain;
```

```
import javax.servlet.ServletRequest;
import javax.servlet.ServletResponse;
```

Spring Boot 3 uses the `jakarta.servlet` package:

```
import jakarta.servlet.Filter;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletRequest;
import jakarta.servlet.ServletResponse;
```

For Spring Boot 3, use:

```
import jakarta.servlet.Filter;
```

Do not use:

```
import javax.servlet.Filter;
```

---

7. Why Does `Filter` Use `ServletRequest` and `ServletResponse`?

The standard Filter interface uses generic Servlet abstractions:

```
ServletRequest
ServletResponse
```

These types are not limited to HTTP-specific operations.

In a normal Spring Boot web application, requests are HTTP requests. Therefore, they can be cast to HTTP-specific types.

```
HttpServletRequest httpRequest =
        (HttpServletRequest) request;
```

```
HttpServletResponse httpResponse =
        (HttpServletResponse) response;
```

After casting, we can access HTTP-specific information.

```
httpRequest.getMethod();
httpRequest.getRequestURI();
httpRequest.getHeader("Authorization");

httpResponse.setStatus(401);
httpResponse.setHeader(
        "X-App-Name",
        "Student API"
);
```

---

8. Role of `DispatcherServlet`

`DispatcherServlet` is the central Servlet of Spring MVC. It coordinates the MVC request-processing flow.

Its responsibilities include:

- Finding the correct controller method
- Resolving controller method arguments
- Invoking the controller
- Handling controller return values
- Converting Java objects into JSON
- Coordinating exception handling
- Producing the HTTP response

Filters execute before this Spring MVC processing begins.

```
Tomcat
  ↓
Registered Filters
```

```
  ↓
DispatcherServlet
  ↓
Controller Mapping and Execution
```

Does a Filter know which controller method will execute?

At the beginning of Filter execution, Spring MVC has usually not yet selected the controller method.

A Filter can directly access information such as:

- HTTP method
- Request URI
- Query parameters
- Headers
- Cookies
- Remote address
- Request body stream

However, it does not naturally work with controller metadata such as:

```
StudentController
getStudentById()
@GetMapping("/api/students/{id}")
```

Logic that depends on the selected controller or handler is generally better suited to a Spring MVC Interceptor.

---

9. Creating a Custom Filter

A basic custom Filter can be created by implementing the `Filter` interface and registering the class as a Spring bean.

```
package in.strikes.filterdemo.filter;
```

```java
import jakarta.servlet.Filter;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.ServletRequest;
import jakarta.servlet.ServletResponse;
import org.springframework.stereotype.Component;

import java.io.IOException;

@Component
public class LoggingFilter implements Filter {

    @Override
    public void doFilter(
            ServletRequest request,
            ServletResponse response,
            FilterChain chain
    ) throws IOException, ServletException {

        System.out.println(
                "Request entered the Filter"
        );

        chain.doFilter(request, response);

        System.out.println(
                "Response passed through the Filter"
        );
    }
}
```

What does the Filter receive?

Parameter	Meaning
`request`	The incoming request

Parameter	Meaning
`response`	The outgoing response
`chain`	The remaining Filters and final target Servlet

Before-chain logic

```
System.out.println(
        "Request entered the Filter"
);
```

This statement runs before the request reaches the next Filter or `DispatcherServlet`.

Continuing the request

```
chain.doFilter(request, response);
```

This passes the request to:

- The next Filter, or
- `DispatcherServlet` when the current Filter is the final Filter

After-chain logic

```
System.out.println(
        "Response passed through the Filter"
);
```

This statement runs after downstream processing returns.

---

10. Understanding `doFilter()`

The main method of a Filter is:

```
void doFilter(
        ServletRequest request,
        ServletResponse response,
```

```
        FilterChain chain
) throws IOException, ServletException;
```

`ServletRequest request`

The request object provides general request information.

```
request.getParameter("name");
request.getRemoteAddr();
request.getContentType();
```

For HTTP-specific information:

```
HttpServletRequest httpRequest =
        (HttpServletRequest) request;
```

Useful HTTP request methods include:

```
httpRequest.getMethod();
httpRequest.getRequestURI();
httpRequest.getRequestURL();
httpRequest.getHeader("Authorization");
httpRequest.getCookies();
httpRequest.getQueryString();
httpRequest.getSession();
```

`ServletResponse response`

The response object represents the outgoing response.

```
HttpServletResponse httpResponse =
        (HttpServletResponse) response;
```

It can be used to:

```
httpResponse.setStatus(401);
httpResponse.setHeader("X-App", "Demo");
httpResponse.setContentType("application/json");
httpResponse.getWriter().write("...");
```

`FilterChain chain`

`FilterChain` represents everything that remains after the current Filter.

This may include:

- Other registered Filters
- `DispatcherServlet`
- Controller execution
- Service and repository calls
- Response conversion

The current Filter does not need to know which exact component comes next. It only calls:

```
chain.doFilter(request, response);
```

---

11. Role of `FilterChain`

Suppose an application contains three Filters:

```
AuthenticationFilter
LoggingFilter
TimingFilter
```

The request-processing chain may look like this:

```
AuthenticationFilter
        ↓
LoggingFilter
        ↓
```

```
TimingFilter
        ↓
DispatcherServlet
```

Every Filter has two choices.

Continue the request

```
chain.doFilter(request, response);
```

Stop the request

```
response.setStatus(
        HttpServletResponse.SC_UNAUTHORIZED
);

return;
```

> If a Filter does not call `chain.doFilter(...)`, the request does not continue to the remaining Filters or controller.

---

12. Request and Response Execution Order

Filters surround the remaining request-processing chain.

Suppose two Filters are registered:

```
AuthenticationFilter → LoggingFilter → Controller
```

The incoming request executes in normal order, while the response-side logic executes in reverse order.

Authentication Filter

```java
@Component
@Order(1)
public class AuthenticationFilter implements Filter {

    @Override
    public void doFilter(
            ServletRequest request,
            ServletResponse response,
            FilterChain chain
    ) throws IOException, ServletException {

        System.out.println(
                "Authentication before"
        );

        chain.doFilter(request, response);

        System.out.println(
                "Authentication after"
        );
    }
}
```

Logging Filter

```java
@Component
@Order(2)
public class LoggingFilter implements Filter {

    @Override
    public void doFilter(
            ServletRequest request,
            ServletResponse response,
            FilterChain chain
```

```
    ) throws IOException, ServletException {

        System.out.println("Logging before");

        chain.doFilter(request, response);

        System.out.println("Logging after");
    }
}
```


Controller

```
@GetMapping("/hello")
public String hello() {
    System.out.println("Controller");

    return "Hello";
}
```

Output:

```
Authentication before
Logging before
Controller
Logging after
Authentication after
```

The execution can be visualized as:

```
AuthenticationFilter before
        ↓
LoggingFilter before
        ↓
Controller
        ↓
LoggingFilter after
```

```
        ↓
AuthenticationFilter after
```

13. Reading HTTP Request and Response Information

A practical Filter usually casts the generic Servlet objects to HTTP-specific objects.

```java
@Component
public class RequestLoggingFilter implements Filter {

    @Override
    public void doFilter(
            ServletRequest request,
            ServletResponse response,
            FilterChain chain
    ) throws IOException, ServletException {

        HttpServletRequest httpRequest =
                (HttpServletRequest) request;

        HttpServletResponse httpResponse =
                (HttpServletResponse) response;

        System.out.println(
                "Incoming request: "
                        + httpRequest.getMethod()
                        + " "
                        + httpRequest.getRequestURI()
        );

        chain.doFilter(
                httpRequest,
                httpResponse
        );
```

```
        System.out.println(
                "Response status: "
                        + httpResponse.getStatus()
        );
    }
}
```

Example output:

```
Incoming request: GET /api/students
Response status: 200
```

---

14. Common Use Cases of Filters

14.1 Request Logging

A Filter can log common request information for every API.

```
String method = httpRequest.getMethod();

String uri = httpRequest.getRequestURI();

String remoteAddress =
        httpRequest.getRemoteAddr();

String userAgent =
        httpRequest.getHeader("User-Agent");
```

Example log:

```
Method: GET
URI: /api/students
Remote IP: 192.168.1.10
User-Agent: PostmanRuntime
```

14.2 Authentication Token Inspection

A Filter can inspect an authentication header.

```
String token =
        httpRequest.getHeader("Authorization");
```

It may then:

- Check whether the token exists
- Validate the token
- Reject invalid requests
- Extract user information
- Populate security-related context

Spring Security itself relies heavily on a chain of Filters.

In a real secured application, authentication should normally be integrated with Spring Security rather than implemented through an unrelated custom Filter.

14.3 Character Encoding

A Filter can enforce consistent request and response encoding.

```
request.setCharacterEncoding("UTF-8");
response.setCharacterEncoding("UTF-8");
```

Spring also provides built-in support such as `CharacterEncodingFilter`.

14.4 Common Response Headers

A Filter can add headers that should appear in multiple responses.

```
httpResponse.setHeader(
        "X-Application-Name",
        "Student Service"
```

```
);

httpResponse.setHeader(
        "X-Content-Type-Options",
        "nosniff"
);
```


Common purposes include:

- Security headers
- Request tracking
- Application metadata
- Cache control

14.5 Measuring Request Duration

A Filter can measure the time spent in the downstream request-processing chain.

```
long startTime =
        System.currentTimeMillis();

chain.doFilter(request, response);

long duration =
        System.currentTimeMillis() - startTime;

System.out.println(
        "Duration: " + duration + " ms"
);
```

This generally includes processing performed by:

- Remaining Filters
- `DispatcherServlet`
- Interceptors

- Controller
- Service
- Repository
- Response conversion

The exact measurement depends on the position of the Filter and how the response is handled.

14.6 Blocking Unwanted Requests

A Filter can reject a request before it reaches Spring MVC.

```
String apiKey =
        httpRequest.getHeader("X-API-KEY");

if (apiKey == null) {
    httpResponse.setStatus(
            HttpServletResponse.SC_UNAUTHORIZED
    );

    return;
}

chain.doFilter(request, response);
```

The `return` statement stops the current Filter method.

Because `chain.doFilter(...)` was not called, the controller does not execute.

14.7 Correlation or Trace IDs

A Filter can generate a unique ID for every request.

```
String requestId =
        UUID.randomUUID().toString();
```

```
httpResponse.setHeader(
        "X-Request-Id",
        requestId
);
```

The same ID can be included in:

- Application logs
- Response headers
- Downstream service calls
- Error reports

This makes it easier to trace one request across multiple log entries or services.

---

15. Blocking a Request and Returning JSON

A Filter can directly create an error response without allowing the request to reach the controller.

```
package in.strikes.filterdemo.filter;

import jakarta.servlet.Filter;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.ServletRequest;
import jakarta.servlet.ServletResponse;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.stereotype.Component;

import java.io.IOException;

@Component
public class ApiKeyFilter implements Filter {

    @Override
```

```
    public void doFilter(
            ServletRequest request,
            ServletResponse response,
            FilterChain chain
    ) throws IOException, ServletException {

        HttpServletRequest httpRequest =
                (HttpServletRequest) request;

        HttpServletResponse httpResponse =
                (HttpServletResponse) response;

        String apiKey =
                httpRequest.getHeader("X-API-KEY");

        if (apiKey == null
                || !apiKey.equals("secret123")) {

            httpResponse.setStatus(
                    HttpServletResponse.SC_UNAUTHORIZED
            );

            httpResponse.setContentType(
                    "application/json"
            );

            httpResponse.setCharacterEncoding(
                    "UTF-8"
            );

            httpResponse.getWriter().write(
                    """
                    {
                      "message": "Invalid or missing API key"
                    }
                    """
```

```
                );

                return;
            }

            chain.doFilter(request, response);
        }
    }
```

Request flow when the API key is valid

```
API key is valid
      ↓
chain.doFilter(...)
      ↓
Request continues
```

Request flow when the API key is invalid

```
API key is missing or invalid
      ↓
Write a 401 response
      ↓
Return without calling chain.doFilter(...)
      ↓
Request stops
```

> The hard-coded key in this example is only for understanding Filter behaviour. Real authentication should use a proper security mechanism.

---

16. Using `try-finally` for Timing and Cleanup

Consider this code:

```
chain.doFilter(request, response);
```

```
System.out.println("Request completed");
```

If downstream processing throws an exception, the statement after `chain.doFilter(...)` may not execute normally.

A safer pattern for timing, cleanup, or final logging is:

```
long startTime =
        System.currentTimeMillis();

try {
    chain.doFilter(request, response);
} finally {
    long duration =
            System.currentTimeMillis() - startTime;

    System.out.println(
            "Duration: " + duration + " ms"
    );
}
```

The `finally` block executes whether downstream processing completes normally or throws an exception.

---

17. Complete Request Logging Filter

The following Filter logs:

- HTTP method
- Request path
- Query string
- Remote IP address
- Response status
- Total duration

```java
package in.strikes.filterdemo.filter;

import jakarta.servlet.Filter;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.ServletRequest;
import jakarta.servlet.ServletResponse;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Component;

import java.io.IOException;

@Component
public class RequestLoggingFilter implements Filter {

    private static final Logger logger =
            LoggerFactory.getLogger(
                    RequestLoggingFilter.class
            );

    @Override
    public void doFilter(
            ServletRequest request,
            ServletResponse response,
            FilterChain chain
    ) throws IOException, ServletException {

        HttpServletRequest httpRequest =
                (HttpServletRequest) request;

        HttpServletResponse httpResponse =
                (HttpServletResponse) response;
```

```
        long startTime =
                System.currentTimeMillis();

        String method =
                httpRequest.getMethod();

        String requestUri =
                httpRequest.getRequestURI();

        String queryString =
                httpRequest.getQueryString();

        String remoteAddress =
                httpRequest.getRemoteAddr();

        String completePath =
                queryString == null
                        ? requestUri
                        : requestUri
                        + "?"
                        + queryString;

        logger.info(
                "Incoming request: "
                        + "method={}, path={}, ip={}",
                method,
                completePath,
                remoteAddress
        );

        try {
            chain.doFilter(request, response);
        } finally {
            long duration =
                    System.currentTimeMillis()
```

```
                        - startTime;

            logger.info(
                    "Request completed: "
                            + "method={}, path={}, "
                            + "status={}, duration={}ms",
                    method,
                    completePath,
                    httpResponse.getStatus(),
                    duration
            );
        }
    }
}
```

Using a logging framework such as SLF4J is preferred over `System.out.println()` in real applications because logs can be configured, filtered, stored, and integrated with monitoring tools.

18. Filter Ordering with `@Order`

When multiple Filters are registered, their order affects request and response processing.

```
@Component
@Order(1)
public class FirstFilter implements Filter {
    // Filter logic
}
```

```
@Component
@Order(2)
public class SecondFilter implements Filter {
```

```
    // Filter logic
}
```

For an incoming request:

```
FirstFilter
    ↓
SecondFilter
    ↓
DispatcherServlet
```

For the returning response:

```
DispatcherServlet
    ↓
SecondFilter
    ↓
FirstFilter
```

What happens without `@Order`?

A Filter registered as a Spring bean still executes.

However, when several Filters depend on a particular sequence, the application should define their order explicitly instead of relying on an observed default order.

---

19. Where Should Response Logic Be Placed?

Consider adding a response header after downstream processing:

```
chain.doFilter(request, response);

httpResponse.setHeader(
        "X-Test",
        "value"
);
```

This may work when the response has not yet been committed.

However, once the response body or headers have been flushed, changing the status or headers may be too late.

Headers that must definitely be present should generally be added before continuing the chain.

```
httpResponse.setHeader(
        "X-Test",
        "value"
);

chain.doFilter(request, response);
```

Good candidates for pre-processing

- Request validation
- Authentication checks
- Required response headers
- Character encoding
- Request setup

Good candidates for post-processing

- Logging the final response status
- Measuring duration
- Metrics collection
- Cleanup operations

---

20. Logging Safety

Filters can access sensitive request information, but not every value should be written to logs.

Avoid logging:

- Authorization tokens

- Session cookies
- API keys
- Passwords
- Credit card information
- Sensitive personal information

Logging entire request headers or bodies without filtering can create serious security and privacy problems.

> Log only the information required for debugging, auditing, or monitoring, and mask sensitive values wherever necessary.

---

21. Filter Responsibilities at a Glance

Requirement	Is a Filter suitable?
Log all incoming HTTP requests	Yes
Read or validate request headers	Yes
Add a common response header	Yes
Block a request before Spring MVC	Yes
Apply character encoding	Yes
Measure overall request duration	Yes
Work with the selected controller method	Usually no; prefer an Interceptor
Implement full application security manually	No; prefer Spring Security
Execute business logic for one API	No; keep it in controller or service layers

---

22. Key Takeaways

- Filters belong to the Servlet layer.
- They execute before Spring MVC's `DispatcherServlet`.
- A Filter can run logic before and after downstream request processing.
- `chain.doFilter(request, response)` continues the request.

- Returning without calling `chain.doFilter(...)` stops the request.
- Incoming Filter logic executes in order.
- Response-side Filter logic executes in reverse order.
- `HttpServletRequest` and `HttpServletResponse` provide HTTP-specific operations.
- Filters are suitable for logging, headers, encoding, request timing, trace IDs, and early request validation.
- Filter order should be explicitly defined when the sequence matters.
- Sensitive headers, tokens, and personal data should not be logged.
- Authentication in production applications should normally be handled through Spring Security.

Final Mental Model

```
Request enters the server
        ↓
Filter performs pre-processing
        ↓
Filter either blocks the request
or calls chain.doFilter(...)
        ↓
Remaining Filters and DispatcherServlet execute
        ↓
Controller, Service, and Repository execute
        ↓
Response returns through the Filter chain
        ↓
Filter performs post-processing
        ↓
Response reaches the client
```

A Filter is a reusable HTTP-level checkpoint placed around the remaining request-processing flow.